Digital Coursework

1 Task1.1

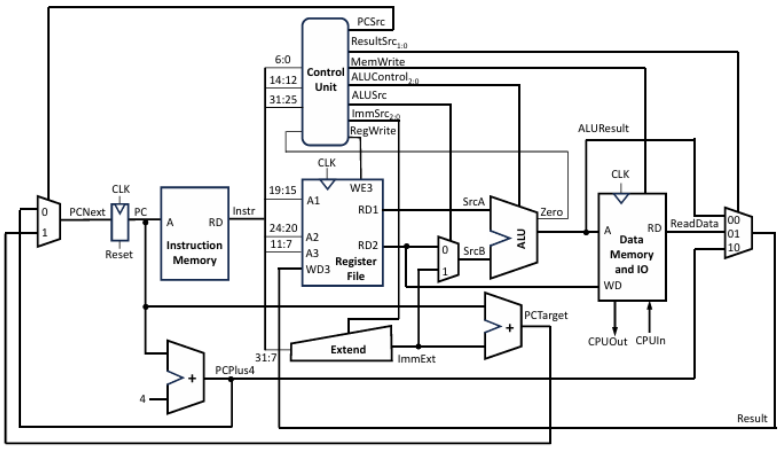

Figure 1: RISC-V microarchitecture

Fig. 1. Original circuit.

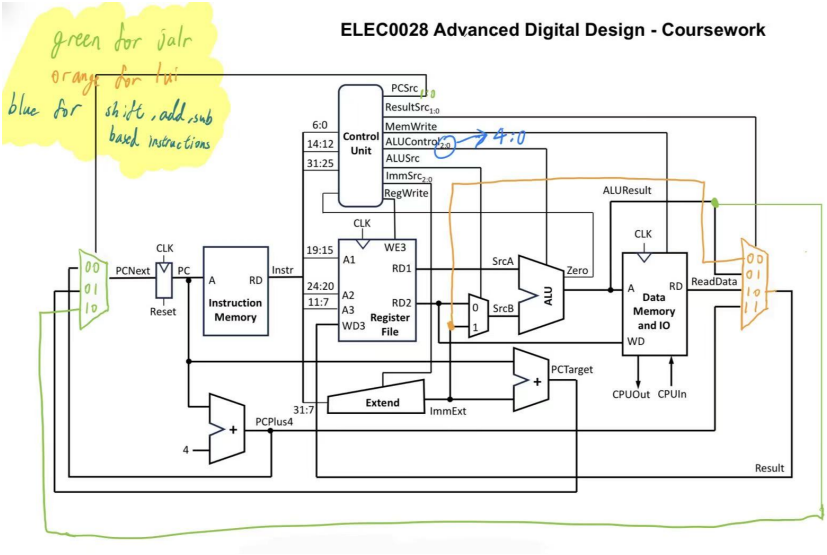

Fig. 2. Redesign of the original circuit.

Author's Contact Information:

1.1 Problems of the Original Microarchitecture

The original microarchitecture shown in Figure 1 cannot support all RV32I instructions due to several structural limitations. The key issues and corresponding modifications (shown in Figure 2) are summarised below.

1.1.1 Limitation 1: Missing PC Input for `jalr`. The PC multiplexer only selects between $PC + 4$ and $PC + \text{Imm}$, and does not support the $rs1 + imm$ operation required by `jalr`. As a result, `jalr` cannot be executed.

To resolve this, the PC selection logic is extended by introducing a third input from `ALUResult`, which is $rs1 + imm$, and expanding `PCSrc` to 2 bits.

1.1.2 Limitation 2: Immediate Cannot Be Written Back (`lui`*).* The output of the Extend unit (`ImmExt`) is not connected to the write-back path, and therefore cannot be stored in the register file. This prevents execution of the `lui` instruction.

This is addressed by adding `ImmExt` as an additional input to the Result multiplexer and extending `ResultSrc` accordingly.

1.1.3 Limitation 3: Insufficient ALU Control Width. The ALU requires a 5-bit control signal to support all operations, but only a 3-bit control signal is provided in the original design. This leads to incomplete or incorrect execution of multiple instructions, including arithmetic operations (`add`, `sub`, `addi`), shift operations (`sll`, `srl`, `slli`, `srli`), comparison operations (`slt`, `slti`), memory access instructions (`lw`, `sw`), and control flow instructions (`beq`, `bne`, `blt`, `bge`, `jalr`). In particular, all operations that rely on addition/subtraction or shift functionality may behave incorrectly due to insufficient ALU control signals.

To fix this, the `ALUControl` signal is expanded from 3 bits to 5 bits, allowing full control of all ALU functionalities.

1.2 Justification for Not Using the Negative Signal

Although the ALU shown in Figure 2 provides both `Zero` and `Negative` outputs, the `Negative` signal is not required in the redesigned processor. The reason is that all comparison-related instructions in the required instruction subset can already be implemented using the `SLT` operation together with the `Zero` signal. :contentReference[oaicite:2]index=2

For `slt` and `slti`, the ALU directly produces a result of either 1 or 0, which is then written back to the destination register. Therefore, these instructions do not require a separate sign output.

For branch instructions, the `ALURESULT` does not need to be written back, so only the branch decision matters. This means the `SLT` result can be used indirectly through the `Zero` flag.

For `blt`, the ALU is configured to perform the `SLT` operation:

- if $rs1 < rs2$, then the SLT result is 1, so $\texttt{Zero} = 0$, and the branch should be taken;
- if $rs1 \geq rs2$, then the SLT result is 0, so $\texttt{Zero} = 1$, and the branch should not be taken.

Therefore, for `blt`, the branch decision can be implemented using:

$$\texttt{PCSrc} = \{\texttt{0}, \sim \texttt{Zero}\}$$

Similarly, for `bge`:

- if $rs1 < rs2$, then the SLT result is 1, so $\texttt{Zero} = 0$, and the branch should not be taken;
- if $rs1 \geq rs2$, then the SLT result is 0, so $\texttt{Zero} = 1$, and the branch should be taken.

Hence, for `bge`, the branch decision can be implemented as:

$$\texttt{PCSrc} = \{\texttt{0}, \texttt{Zero}\}$$

This shows that `Zero` is sufficient to distinguish both branch conditions once the ALU is operating in `SLT` mode. As a result, the `Negative` output does not provide any essential additional information for the supported instruction set.

In summary, the `Negative` signal can be omitted. In the following sections, the processor design presented in this report consistently adopts the implementation without the `Negative` signal. The simulation results in Section **??** confirm that all required instructions operate correctly under this design, demonstrating that the `Negative` output is not necessary.

2 Control Signal Specification

instr	RegWrite	ImmSrc	ALUSrc	ALUControl	MemWrite	ResultSrc	PCSrc
add	1	XXX	0	X0X10	0	01	00
sub	1	XXX	0	X1X10	0	01	00
or	1	XXX	0	XX111	0	01	00
and	1	XXX	0	XX011	0	01	00
sll	1	XXX	0	0XX00	0	01	00
srl	1	XXX	0	1XX00	0	01	00
slt	1	XXX	0	X1X01	0	01	00
addi	1	000	1	X0X10	0	01	00
ori	1	000	1	XX111	0	01	00
andi	1	000	1	XX011	0	01	00
slli	1	000	1	0XX00	0	01	00
srli	1	000	1	1XX00	0	01	00
slti	1	000	1	X1X01	0	01	00
lw	1	000	1	X0X10	0	10	00
sw	0	001	1	X0X10	1	XX	00
beq	0	010	0	X1X10	0	XX	{0,Zero}
bne	0	010	0	X1X10	0	XX	{0,~Zero}
blt	0	010	0	X1X01	0	XX	{0,~Zero}
bge	0	010	0	X1X01	0	XX	{0,Zero}
jal	1	100	X	XXXXX	0	11	01
jalr	1	000	1	X0X10	0	11	10
lui	1	011	X	XXXXX	0	00	00

Fig. 3. Control signals for supported instructions.

Figure 3 summarises the control signals for all supported instructions. Each signal is defined as follows.

2.1 Meaning of Control Signals

RegWrite enables writing the Result back to register `rd`.

ImmSrc selects the type of immediate generated by the Extend unit:

- 000: I-type
- 001: S-type
- 010: B-type
- 011: U-type
- 100: J-type

ALUSrc selects the second ALU operand:

- 0: register value (RD2)
- 1: immediate (ImmExt)

ALUControl[4:0] determines the ALU operation:

- Bit [4]: shift direction (0 = left, 1 = right)
- Bit [3]: arithmetic (0 = add, 1 = sub)
- Bit [2]: logic (0 = and, 1 = or)

- Bits [1:0]:
  - 00: shift
  - 01: set-less-than
  - 10: arithmetic
  - 11: logic

MemWrite enables writing to data memory.
ResultSrc selects the write-back source:

- 00: ImmExt
- 01: ALUResult
- 10: ReadData
- 11: PCPlus4

PCSrc selects the next PC:

- 00: $PC + 4$
- 01: PC + ImmExt
- 10: ALUResult

For branch instructions, symbolic forms are used:

- {0,Zero}: branch when Zero = 1
- {0,~Zero}: branch when Zero = 0

2.2 Instruction-by-Instruction Explanation

2.2.1 R-type Arithmetic and Logic Instructions (`add`*,* `sub`*,* `and`*,* `or`*).* These instructions use two register operands, so `ALUSrc = 0` and `RegWrite = 1`. The result is written back from the ALU (`ResultSrc = 01`), and execution proceeds sequentially (`PCSrc = 00`).

The difference lies in `ALUControl`: `add` uses addition (`X0X10`), `sub` uses subtraction (`X1X10`), while `and` and `or` use the logic path (`XX011` and `XX111`, respectively).

2.2.2 R-type Shift and Comparison Instructions (`sll`*,* `srl`*,* `slt`*).* These instructions also use register operands (`ALUSrc = 0`, `RegWrite = 1`, `ResultSrc = 01`).

Shift instructions (`sll`, `srl`) select the shift path (`[1:0] = 00`), with bit [4] determining direction (left or right).

`slt` selects the comparison path (`[1:0] = 01`), producing a result of 0 or 1.

2.2.3 I-type Arithmetic and Logic Instructions (`addi`*,* `andi`*,* `ori`*).* These instructions use an immediate operand, so `ImmSrc = 000` and `ALUSrc = 1`. The ALU result is written back to the register file.

The ALU operation matches their R-type counterparts: addition for `addi`, AND for `andi`, and OR for `ori`.

2.2.4 I-type Shift and Comparison Instructions (`slli`*,* `srli`*,* `slti`*).* These instructions also use `ImmExt` as the second operand (`ALUSrc = 1`).

Shift instructions select the shift path, using the lower 5 bits of the immediate as the shift amount.

`slti` performs comparison using the SLT path and writes the result (0 or 1) to the destination register.

2.2.5 Memory Instructions (`lw`*,* `sw`*).* Both instructions compute the effective address using addition (`rs1 + ImmExt`), so `ALUSrc = 1` and `ALUControl` selects addition.

For `lw`, `RegWrite = 1` and `ResultSrc = 10` so that data read from memory is written to the register file.

For `sw`, `RegWrite = 0` and `MemWrite = 1`, storing the value of `rs2` into memory.

2.2.6 Branch Instructions (`beq`*,* `bne`*,* `blt`*,* `bge`*).* These instructions do not write to the register file (`RegWrite = 0`) and use B-type immediates.

`beq` and `bne` use subtraction to compare operands, with branching controlled by the `Zero` signal.

`blt` and `bge` use the SLT operation. The branch decision is determined from the `Zero` flag: `blt` branches when `Zero = 0`, while `bge` branches when `Zero = 1`.

More detailed reasoning for this design choice is discussed in Section 1.2.

2.2.7 Jump Instructions (`jal`*,* `jalr`*).* Both instructions write $PC + 4$ to the destination register (`ResultSrc = 11`, `RegWrite = 1`).

`jal` uses a J-type immediate and sets `PCSrc = 01`, selecting PC + ImmExt.

`jalr` uses an I-type immediate and computes the target address using the ALU (`rs1 + ImmExt`), so `PCSrc = 10`.

2.2.8 Immediate Upper Instruction (`lui`*).* `lui` directly writes the U-type immediate to the destination register, so `ResultSrc = 00`. The ALU is not used, and execution proceeds sequentially.